

Python Interview Prep — Batch E14 Notes

Your 171-page PDF covers **Python fundamentals** in this order: History → Language types → Python features → Keywords/Identifiers → Data types (String, Tuple, List, Dictionary, Set) → Type conversion. Below are the important points likely to be asked, with short recall tips for each.

1. History of Python

Key points

- Created by **Guido van Rossum**, started in **1989**, released in **1991**.
- Built in the **Netherlands**, during his Christmas holidays.
- Named after "**Monty Python's Flying Circus**" — not the snake.

Recall tip: "Guido, Netherlands, '89-'91, Monty Python" → GNMM.

2. What is Python / Language Hierarchy

Key points

- Python = general-purpose, **high-level**, interpreted programming language.
- Language hierarchy (low → high): **Machine language (binary)** → **Assembly language (mnemonics)** → **High-level language (English-like, e.g., Python, Java, C++)**.
- A **programming language** = medium of communication between humans and machines.

Recall tip: "Machine talks in 0/1, Assembly talks in codes, High-level talks like us."

3. Key Features of Python (very commonly asked)

Feature	Meaning
Interpreted	Code runs line by line, no separate compile step needed by the user
Dynamically typed	No need to declare variable type — decided at runtime based on value
Platform independent	<code>.py</code> file runs on any OS without rewriting
Simple & readable syntax	Less code, closer to English

Feature	Meaning
Free & open source	
Extensive libraries	AI, Data Science, Web Dev, Automation, Testing
Supports dynamic memory management	

Recall tip: "I DPS FEd" → Interpreted, Dynamic typing, Platform independent, Simple syntax, Free/open, Extensive libraries, Dynamic memory.

Common interview question: *Difference between statically typed and dynamically typed language?* → Statically typed = type declared before use (C, Java). Dynamically typed = type decided automatically at runtime based on the assigned value (Python).

4. Keywords vs Identifiers

Keywords

- Reserved/predefined words with special meaning (e.g., `if`, `else`, `def`, `class`, `True`, `None`).
- Case-sensitive (`True` ✓, `true` ✗).
- Check with: `keyword.iskeyword("True")`.
- Total keywords in Python 3.x = 35 (approx., varies by version — mention "around 33-36" if version isn't specified).

Identifiers

- Name given to any programmable entity — variable, function, class, etc.
- Rules:
 1. Can contain letters, digits, underscore — **cannot start with a digit.**
 2. Cannot use special characters like `@, #, %, -, space`.
 3. Cannot be a keyword.
 4. Case-sensitive.

Recall tip: "Keywords are Python's own words; Identifiers are the names *you* create — and your names must follow the 4 rules (No digit-start, No special chars, No keyword, Case matters)."

PEP 8 = Python's official style guide (naming conventions, indentation, etc.) — commonly asked as a one-liner: *"PEP8 is Python's coding style guide."*

5. Variables

Key points

- A name pointing to a memory location holding a value.
- No declaration needed — dynamic typing.
- Naming convention: `snake_case` (preferred in Python) vs `camelCase` (used in Java/JS).
- `type()` → tells the data type of a value/variable.

Recall tip: "Python variables don't wear a fixed uniform (type) — they change based on what they're holding."

6. Data Types — the big interview zone

Category split (most commonly tested)

- **Single-value (scalar) types:** `int`, `float`, `str`, `bool`, `complex`
- **Multi-value (collection) types:** `list`, `tuple`, `dict`, `set`

Recall tip: "LTDS" = List, Tuple, Dict, Set → the 4 collection types.

String

- Sequence of characters; created using `' '`, `" "`, or `''' '''` (triple quotes for multi-line/docstrings).
- **Ordered, immutable, allows duplicates.**
- Indexing: forward `[0, 1, 2...]`, backward `[-1, -2, -3...]`
- Slicing syntax: `[var[start:end:step]]` — default start=0, end=length, step=1.
- Important methods to remember: `lower()`, `upper()`, `title()`, `strip()`, `split()`, `join()`, `find()`, `index()`, `count()`, `replace()`, `startswith()`, `endswith()`, `isalnum()`, `isdigit()`, `isupper()`, `isspace()`, `removeprefix()`, `removesuffix()`.
- **find() vs index():** both return position of substring; `find()` returns `-1` if not found, `index()` raises `ValueError`. (Classic interview question!)

Recall tip: "find is Forgiving (-1), index Isn't (Error)."

Tuple

- Ordered, **immutable** collection; created with `()` or `tuple()`.
- Allows duplicates.
- Supports indexing/slicing like lists, but **no add/remove methods** (immutable) — only `count()` and `index()`.

Recall tip: "Tuple = List's strict twin — same order & access, but locked (immutable)."

List

- Ordered, **mutable** collection; created with `[]`.
- Allows duplicates, allows nested lists.
- Common methods: `append()`, `extend()`, `insert()`, `pop()`, `remove()`, `del`, `count()`, `index()`, `sort()`.
- **append() vs extend()**: `append()` adds a single item (even a list becomes one nested item); `extend()` unpacks an iterable and adds each item individually. (Very frequently asked!)
- **pop() vs remove() vs del**: `pop(index)` removes by position (returns the value); `remove(value)` removes by value; `del` deletes by index or the whole list (returns nothing).
- **Copying a list — 3 types (important!):**
 - `copy()` / normal assignment issue → shallow copy shares nested objects.
 - **Shallow copy**: copies outer list, but nested objects still reference the same memory.
 - **Deep copy** (`from copy import deepcopy`): copies everything recursively — no shared references.

Recall tip: "Shallow = skin-deep copy (inner lists still linked); Deep copy = fully cloned, nothing shared."

Dictionary

- Collection of **key-value pairs**; created with `{}` or `dict()`.
- Keys must be **unique** and **immutable** (string, number, tuple); values can be any type & can repeat.
- Access: `dict[key]` (raises `KeyError` if missing) vs `dict.get(key)` (returns `None` if missing — safer).
- Common methods: `keys()`, `values()`, `items()`, `update()`, `pop()`, `popitem()`, `setdefault()`, `fromkeys()`.
- Merging dicts: `dict1.update(dict2)` or `{**dict1, **dict2}`.

Recall tip: "`get()` is the **safe** way to access — no crash on missing key; `dict[key]` is the **risky** way."

Set

- Unordered, **mutable**, collection of **unique** elements (no duplicates); created with `{}` or `set()`.
- **Not subscriptable** — no indexing/slicing (`s[0]` → error) because it's unordered.

- Set operations (classic Venn-diagram questions): `union()`, `intersection()`, `difference()`, `symmetric_difference()`, `issubset()`, `issuperset()`, `isdisjoint()`.
- Add/remove: `add()`, `update()` (add multiple), `remove()` (error if missing), `discard()` (no error if missing), `pop()` (removes random element).

Recall tip: "Set = Squad with no repeats, no fixed order. `remove` complains if missing, `discard` doesn't."

7. Type Conversion

- `int()`, `float()`, `str()`, `list()`, `tuple()`, `set()`, `dict()`, `complex()`, `bool()` convert between types.
- Note: converting to `dict()` needs key-value structure (e.g., a list of pairs), not any random iterable.
- `isinstance(variable, type)` → checks type; can check multiple types with a tuple: `isinstance(x, (int, float))`.

Recall tip: "`isinstance` checks identity like a bouncer checking IDs — can check against a single type or a guest-list (tuple) of types."

★ Top 10 "Definitely Will Be Asked" Questions

1. Who developed Python and when? (Guido van Rossum, 1989/1991)
2. Why is Python called a dynamically typed language?
3. Difference between list, tuple, set, and dictionary.
4. Difference between `append()` and `extend()`.
5. Difference between shallow copy and deep copy.
6. Difference between `find()` and `index()`.
7. Difference between `remove()` and `discard()` in sets.
8. Why can't you slice/index a set?
9. What is a mutable vs immutable data type? (str/tuple = immutable; list/dict/set = mutable)
10. What are Python keywords vs identifiers, and identifier naming rules?

Fast interview recall chain: *Mutable* = can change after creation (list, dict, set) → *Immutable* = cannot (int, float, str, tuple, bool, complex).

Data Type	Ordered?	Mutable?	Duplicates?	Symbol
String	Yes	No	Yes	'...'
Tuple	Yes	No	Yes	(...)
List	Yes	Yes	Yes	[...]
Dictionary	Yes (3.7+)	Yes	Keys: No, Values: Yes	{k:v}
Set	No	Yes	No	{...}

One-line memory hook: *"Strings and Tuples are frozen; Lists, Dicts, and Sets can move."*